

ASTRALINK

Object-Oriented Channel Coding and Data Transmission Simulator

Course Name: Basics of Programming II

Course ID: ENVIAUAA01

Student Name: Mohamed Ayman Swesa

Neptun Code: D044RQ

Lab Group ID: EE16D

Lab Instructor: Dr. Tóth Zoltán

Project Type: Final Project

Submission Date: May 24, 2026

1. Introduction

In digital telecommunications, raw data transmitted across a physical medium is highly susceptible to noise and corruption. To address this problem, I developed AstraLink: an object-oriented C++ simulation of a data transmission pipeline. This project models the full lifecycle of a binary message, exploring how data can be encapsulated, mathematically encoded with Forward Error Correction (FEC) algorithms, subjected to stochastic environmental noise, and successfully decoded at the receiver to maintain data integrity. The primary terminology used throughout this document includes **FEC** (Forward Error Correction), **Syndrome** (the calculated error index during decoding), and **Aliasing** (a false syndrome caused by multi-bit errors).

2. Program Interface

AstraLink is a command-line interface (CLI) application. To start the program, the user must compile the source files using a standard C++ compiler (such as GCC/MinGW) and execute the resulting `main.exe` binary from their terminal. The programming environment is entirely self-contained; no external parameters or flags are required to accompany the execution command. The program terminates automatically and cleanly once a single pipeline transmission completes and the final output is written to disk. The user can also manually terminate the environment at any time using the standard `Ctr l+C` terminal interrupt.

3. Program Execution

From the end-user's perspective, executing the program triggers an interactive console menu. I designed the system to prompt the user to select their desired coding scheme:

- Option 1: Parity Coder (Even Parity)
- Option 2: Hamming (7,4) FEC Coder

Once the user enters their choice, the program seamlessly takes over. It prints status updates to the terminal, indicating when the file is read, displaying the encoded payload before transmission, showing the corrupted payload after it passes through the noisy channel, and finally outputting the corrected, decoded data. If the user enters an invalid menu option, the program catches the error and politely requests a restart.

4. Input and Output

The system relies heavily on the standard `<fstream>` library for data persistence. The program requires an input file named `input.txt` located in the same working directory as the executable. The format of this file must be a continuous string of binary characters (e.g., `10110011`). Non-binary characters will trigger an internal validation exception.

Upon successful execution, the program creates or overwrites an output file named `output.txt` in the same directory. The output format is identical to the input format: a clean, continuous string of the recovered binary data.

5. Program Structure

I structured AstraLink using a highly modular, object-oriented design to separate the data wrapper from the mathematical processors.

5.1 Data Structures: *DataStream*

To enforce encapsulation, I wrapped the raw `std::string` payload inside a `DataStream` class. The payload is strictly private. To facilitate easier file operations without breaking encapsulation, I implemented an inline friend function to overload the `<<` stream insertion operator directly in the header file.

5.2 Modules: *Polymorphic Encoders*

The core logic utilizes an abstract base class, `ChannelCoder`, which enforces `encode()` and `decode()` methods. From this, I derived two specific classes:

- **ParityCoder:** Calculates modulo-2 logic to append a single error-detection bit.
- **HammingCoder:** Slices the data into 4-bit chunks, utilizes bitwise XOR matrices to calculate three overlapping parity bits, and packages them into a 7-bit block. During decoding, it calculates the 3-bit Syndrome to locate and flip corrupted bits.

5.3 Modules: *Stochastic Channel*

The `Channel` class simulates the physical wire. Instead of legacy functions, I utilized the `<random>` library, specifically a hardware-seeded `std::mt19937` Mersenne Twister engine and a uniform real distribution to randomly flip bits based on a configured probability parameter.

6. Examples

If the user provides `input.txt` containing `1011` and selects the Hamming (7,4) coder, the program generates the 7-bit block `0110011`. If the Channel randomly flips the fourth bit (yielding `0111011`), the decoder calculates a Syndrome of 4, safely flips the bit back to 0, strips the parity, and outputs the clean `1011` to `output.txt`.

7. Testing and Verification

Testing was performed locally using GCC. To verify the Hamming matrix logic, I forced the `Channel` class error probability to extreme highs and lows. During high-noise testing, I verified the presence of the Aliasing phenomenon: when the channel flipped two bits simultaneously, the Hamming algorithm falsely identified the wrong error index and corrupted the data further. Exception handling (`try/catch`) was verified by intentionally deleting `input.txt` and entering invalid menu choices; in all cases, the program gracefully caught the `std::runtime_error` and `std::invalid_argument` exceptions without crashing.

8. Improvements and Extensions

A primary shortcoming of the current implementation is its susceptibility to Aliasing under high-noise conditions, as the Hamming (7,4) code only has a Hamming distance of 3. A significant future extension would be implementing an Extended Hamming (8,4) class (SECDED), which adds a global parity bit to detect double-bit errors and prevent false corrections. Additionally, implementing a user prompt to manually set the noise probability at runtime would enhance interactivity.

9. Difficulties Encountered

A major technical hurdle involved C++ linker errors when fulfilling the operator overloading requirement. Initially, writing the full `<<` overload logic directly in the `DataStream.hpp` file caused multiple-definition errors when included in multiple `.cpp` files. I resolved this by researching the One Definition Rule (ODR) and applying the `inline` keyword to the friend function, allowing the compiler to safely embed the logic.

10. Conclusion

The AstraLink simulator successfully fulfills the requirements of the final project by merging advanced object-oriented programming paradigms with practical telecommunications theory. By implementing polymorphism, robust exception handling, dynamic memory management, and file streams, the project demonstrates a comprehensive understanding of sustainable C++ software engineering.

11. References

- [1] Bjarne Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley Professional, 2013.
- [2] "C++ Standard Library Documentation," `cppreference.com`. [Online]. Available: <https://en.cppreference.com/>.